
Task 8: SQL Injection Practical Exploitation (Explained Clearly)

1. Identify Injectable Parameters

This step is about finding **input fields that interact with the database without proper validation**.

Common places attackers test:

- URL parameters (e.g., `id=5`)
- Login forms (username/password fields)
- Search boxes
- Cookies or HTTP headers

Why this matters:

If user input is directly inserted into an SQL query without filtering or parameterization, the database may interpret that input as **SQL code instead of data**.

Example idea (conceptual):

If an application expects a number but crashes or behaves strangely when given unexpected characters, that parameter might be injectable.

2. Run SQLMap

SQLMap is an **automated SQL injection testing tool** used during **authorized penetration testing**.

What it does (at a high level):

- Tests parameters automatically for SQL injection
- Detects the type of database (MySQL, PostgreSQL, etc.)
- Exploits vulnerabilities to extract data if allowed

Why tools like this exist:

Manual testing is slow and error-prone. SQLMap simulates what a real attacker would do, but in a **controlled and legal environment**.

3. Extract Database Names

Once injection is confirmed, an attacker's next goal is **reconnaissance**.

Why databases matter:

- A single server can host multiple databases
- Some databases may contain sensitive or critical information

At this stage, the attacker is essentially asking:

“What databases exist on this server?”

Knowing database names helps attackers decide **where valuable data might live**.

4. Extract Tables

After identifying a target database, the next step is learning its **structure**.

Tables typically include:

- Users
- Orders
- Payments
- Logs
- Admin data

Why tables are important:

Tables define **what kind of data is stored**. A table named `users` or `credentials` is far more interesting than one called `logs`.

This is similar to looking at a filing cabinet and reading the labels before opening drawers.

5. Extract User Data

This is the most dangerous phase.

Common data targeted:

- Usernames
- Password hashes
- Email addresses
- Personal information

Impact:

Even if passwords are hashed, attackers may:

- Crack weak hashes
- Reuse credentials on other platforms
- Sell the data

This step often leads to **data breaches**, identity theft, or account takeovers.

6. Analyze Impact

This step focuses on **what the breach actually means**.

Questions to consider:

- Was sensitive personal data exposed?
- Could accounts be taken over?
- Is the organization violating data protection laws?
- Could attackers modify or delete data?

Potential consequences:

- Financial loss
- Legal penalties (GDPR, HIPAA, etc.)
- Loss of customer trust
- Reputation damage

This is where a “small bug” turns into a **serious business risk**.

7. Document Attack Flow

This is about **clear reporting**, not hacking.

A good attack flow explanation includes:

1. Entry point identified (e.g., vulnerable parameter)
2. Injection confirmed
3. Database enumeration
4. Data extraction
5. Potential escalation

Why documentation matters:

- Helps developers understand what went wrong
- Helps management assess risk
- Provides proof for security audits
- Prevents the same issue from happening again

Clear documentation turns an incident into a **learning opportunity**.

8. Suggest Fixes

This is the most important part

Effective mitigations include:

- **Prepared statements / parameterized queries**
 - Ensures user input is treated as data, not SQL code
 - **Input validation and sanitization**
 - Reject unexpected characters or formats
 - **Least privilege database accounts**
 - The app should not have admin-level DB access
 - **Web Application Firewalls (WAFs)**
 - Can block common SQL injection patterns
 - **Regular security testing**
 - Automated scans + manual code reviews
 - **Error handling**
 - Never expose database errors to users
-